kalman_filter

Release 0.1

Chheng Lydiya

Mar 03, 2026

CONTENTS

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

INTRODUTORY

1.1 Introduction

These notes focus on **Kalman filtering** and **optimal state estimation**, covering both the underlying theory and practical methods for estimating the state of dynamic systems from noisy measurements. Kalman filters are widely used in engineering, robotics, navigation, finance, and many other fields where systems change over time and measurements are uncertain.

1.1.1 Scope of These Notes

The goal of this document is to provide a structured set of notes that:

- Explains the mathematics and intuition behind **state estimation**.
- Covers **linear system models**, Gaussian noise assumptions, and their role in deriving optimal estimates.
- Derives the **Kalman filter equations** from first principles.
- Explores **extensions and variations**, including the Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), and Kalman smoothing techniques.
- Provides examples and derivations to build intuition about uncertainty, prediction, and measurement updating.

These notes are intended for readers who already have a basic understanding of probability and random variables. As such, rather than starting from scalar random variables and elementary probability concepts, the discussion begins at **multiple random variables**. This approach is more relevant for practical systems, where different state components—such as position, velocity, or sensor biases—interact and must be estimated simultaneously.

1.1.2 Why Start at Multiple Random Variables

Real-world systems are rarely one-dimensional. A robot's motion, an aircraft's navigation, or a financial portfolio all involve **interdependent quantities** that cannot be treated independently. Modeling these requires **vector random variables**, which allow us to describe not only the expected value of each component but also the **covariance between components**, capturing how uncertainties are correlated.

By starting here, these notes focus on the mathematical tools and concepts that directly feed into **state-space modeling** and the **Kalman filter**, including:

- **Mean vectors** and **covariance matrices** to represent uncertainty in multiple dimensions.
- **Joint Gaussian distributions** and their properties, which are foundational for deriving Kalman updates.
- **Linear transformations** of random vectors, which appear in system dynamics and measurement models.
- **Conditional distributions**, which are central to updating state estimates when new measurements become available.

1.1.3 Structure and Approach

The notes are organized to gradually build from these foundational concepts toward a full understanding of the Kalman filter:

1. **Multiple Random Variables** – vector notation, covariance, joint Gaussian distributions, and linear transformations.
2. **State-Space Models** – linear system representation, process and measurement models, and noise characterization.
3. **Kalman Filter** – derivation of prediction and update equations, intuition, and properties.
4. **Extensions** – nonlinear systems (EKF, UKF), smoothing, and other variations.
5. **Applications and Examples** – practical scenarios, implementation tips, and illustrative computations.

By framing the notes in this way, readers can focus on the **core ideas of state estimation** without being slowed by material they already know, while still maintaining a complete and coherent understanding of Kalman filtering.

1.1.4 Why These Notes Matter

Kalman filtering is not just a theoretical exercise—it is a **powerful tool** for making sense of uncertain, dynamic systems. Understanding how to model uncertainty, combine predictions with measurements, and optimize estimates is a skill that applies across engineering, data science, and decision-making contexts. These notes are designed to provide both the **theoretical foundation** and **practical insight** to develop that skill.

1.2 Probability

> **Attention**
>
> This section does not cover all probability theory. It focuses on multiple random variables and stochastic processes.

1.2.1 Multiple Random Variables

CDF

If X and Y are random variables (RVs), their cumulative distribution functions (CDFs) are

$$F_X(x) = P\big(X \leq x\big)$$

$$F_Y(y) = P\big(Y \leq y\big)$$

The joint CDF is

$$F_{X,Y}(x,y) = P\big(X \leq x,\ Y \leq y\big)$$

Properties of the joint CDF

$$0 \leq F_{X,Y}(x,y) \leq 1$$

$$\lim_{x \to -\infty} F_{X,Y}(x,y) = 0, \quad \lim_{y \to -\infty} F_{X,Y}(x,y) = 0$$

$$\lim_{x,y \to \infty} F_{X,Y}(x,y) = 1$$

$$extif a \leq b \text{ and } c \leq d, \quad F_{X,Y}(a,c) \leq F_{X,Y}(b,d)$$

$$P\big(a < X \leq b,\ c < Y \leq d\big) = F_{X,Y}(b,d) - F_{X,Y}(a,d) - F_{X,Y}(b,c) + F_{X,Y}(a,c)$$

Joint PDF

When X and Y are (jointly) continuous, the joint probability density function (PDF) is the mixed partial derivative of the joint CDF:

$$f_{X,Y}(x,y) = \frac{\partial^2}{\partial x \partial y} F_{X,Y}(x,y)$$

Equivalently,

$$F_{X,Y}(x,y) = \int_{-\infty}^{x} \int_{-\infty}^{y} f_{X,Y}(u,v)\, dv\, du$$

Properties of the joint PDF

$$f_{X,Y}(x,y) \geq 0$$

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{X,Y}(x,y)\, dx\, dy = 1$$

$$P\big(a < X \leq b,\ c < Y \leq d\big) = \int_{a}^{b} \int_{c}^{d} f_{X,Y}(x,y)\, dy\, dx$$

Marginal PDFs

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\, dy$$

$$f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\, dx$$

For discrete random variables, replace PDFs with PMFs and integrals with sums.

Expected value

The expected value of a function $g(\cdot,\cdot)$ of two RVs is

$$E\big[g(X,Y)\big] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x,y)\, f_{X,Y}(x,y)\, dx\, dy$$

1.2.2 Statistical independence

Random variables X and Y are independent if

$$P\big(X \leq x,\ Y \leq y\big) = P\big(X \leq x\big)\, P\big(Y \leq y\big), \quad \text{for all } x, y$$

From the definition of joint distribution and PDF, this implies

$$F_{X,Y}(x,y) = F_X(x)\, F_Y(y)$$

$$f_{X,Y}(x,y) = f_X(x)\, f_Y(y)$$

Important

The central limit theorem says that the sum of independent RVs tends toward a Gaussian RV, regardless of the PDF of the individual RVs that contribute to the sum.

Covariance and correlation

We define the covariance of two scalar RVs X and Y as

$$C_{XY} = E\big[(X - E[X])(Y - E[Y])\big] = E[XY] - E[X]E[Y]$$

We define the correlation coefficient of two scalar RVs X and Y as

$$\rho = \frac{C_{XY}}{\sigma_X \, \sigma_Y}$$

Important

The correlation is a normalized measure of linear dependence between two RVs X and Y.

- If X and Y are independent, then $\rho = 0$.
- If Y is a linear function of X, then $\rho = \pm 1$.

We define the (unnormalized) cross-correlation of two scalar RVs X and Y as

$$R_{XY} = E[XY]$$

Important

Two RVs are said to be uncorrelated if $R_{XY} = E[X]E[Y]$.

From the definition of independence, if two RVs are independent, then they are uncorrelated. Independence implies uncorrelatedness, but uncorrelatedness does not imply independence. In the special case in which two RVs are Gaussian and uncorrelated, they are also independent.

Two RVs are orthogonal if $R_{XY} = 0$.

If two RVs are uncorrelated, then they are orthogonal only if at least one of them has zero mean. If two RVs are orthogonal, they may or may not be uncorrelated.

Examples

Die rolls

Two rolls of a fair six-sided die are represented by the RVs X and Y.

The two RVs are independent because one roll does not affect the other. Each face has probability $1/6$.

$$E[X] = E[Y] = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5$$

There are 36 equally likely combinations of the two rolls. The (cross-)correlation is

$$R_{XY} = E[XY] = \frac{1}{36} \sum_{i=1}^{6} \sum_{j=1}^{6} ij = 12.25$$

$$E[X]E[Y] = 3.5 \cdot 3.5 = 12.25$$

Thus $E[XY] = E[X]E[Y]$, so X and Y are uncorrelated.

Dependent opposite outcomes

A rigged machine: on the first spin X takes values 1 or -1 with probability $1/2$ each, and the second spin Y is always the opposite (so $Y = -X$). The possible outcomes are $(1, -1)$ and $(-1, 1)$.

$$E[X] = 0, \quad E[Y] = 0$$

$$E[XY] = \frac{(1)(-1) + (-1)(1)}{2} = -1$$

Since $E[XY] \neq E[X]E[Y]$, these variables are correlated; since $E[XY] \neq 0$ they are not orthogonal.

Dependent conditional outcomes

On the first spin X takes values $-1, 0, 1$ equally likely. On the second spin define $Y = 1$ if $X = 0$, otherwise $Y = 0$.

$$E[X] = \frac{-1 + 0 + 1}{3} = 0$$

$$E[Y] = \frac{0 + 1 + 0}{3} = \frac{1}{3}$$

$$E[XY] = \frac{0 + 0 + 0}{3} = 0$$

Here $E[XY] = E[X]E[Y]$, so X and Y are uncorrelated; and $E[XY] = 0$ so they are orthogonal.

Sum of independent RVs

Suppose X and Y are independent RVs, and define $Z = g(X) + h(Y)$. Then

$$\begin{aligned}
E[Z] &= E[g(X) + h(Y)] \\
&= \iint (g(x) + h(y))\, f_{X,Y}(x, y)\, dx\, dy \\
&= \iint (g(x) + h(y))\, f_X(x) f_Y(y)\, dx\, dy \\
&= \int g(x) f_X(x)\, dx \int f_Y(y)\, dy \\
&\quad + \int h(y) f_Y(y)\, dy \int f_X(x)\, dx \\
&= E[g(X)] + E[h(Y)]
\end{aligned}$$

Therefore the mean of the sum of two independent RVs equals the sum of their means:

$$E[X + Y] = E[X] + E[Y]$$

Expected value of the sum of outcomes

Suppose we roll a die twice

We use X and Y to refer to the two rolls of the die

Z represents the sum of the two outcomes

Then $Z = X + Y$

Since X and Y are independent, we have

$$E(Z) = E(X) + E(Y) = 3.5 + 3.5 = 7$$

1.2.3 Multivariate statistics

Given an n-element RV X and an m-element RV Y

X and Y are comlumn vectors

Correlation

Their correlation is defined as

$$\begin{aligned} R_{XY} &= E\left(XY^T\right) \\ &= \begin{bmatrix} E\left(X_1Y_1\right) & \cdots & E\left(X_1Y_m\right) \\ \vdots & \ddots & \vdots \\ E\left(X_nY_1 & \cdots & E\left(X_nY_m\right) \end{bmatrix} \end{aligned}$$

Covariance

Their covariance is defined as

$$\begin{aligned} C_{XY} &= E\left[\left(X - \bar{X}\right)\left(Y - \bar{Y}\right)^T\right] \\ &= E\left(XY^T\right) - \bar{X}\bar{Y}^T \end{aligned}$$

Autocorrelation

The autocorrelation of th n-element RV X is defined as

$$\begin{aligned} R_X &= E\left[XX^T\right] \\ &= \begin{bmatrix} E\left[X_1^2\right] & \cdots & E\left[X_1X_n\right] \\ \vdots & \ddots & \vdots \\ E\left[X_nX_1\right] & \cdots & E\left[X_n^2\right] \end{bmatrix} \end{aligned}$$

Since $E\left(X_iX_j\right) = E\left(X_jX_i\right)$, then $R_X = R_X^T$

Important

An autocorrelation matrix is always symmetric.

For any n-element column vector z, we have

$$\begin{aligned} z^T R_X z &= z^T E\left[XX^T\right] z \\ &= E\left[z^T XX^T z\right] \\ &= E\left[\left(z^T X\right)^2\right] \\ &\geq 0 \end{aligned}$$

Important

An autocorrelation matrix is always positive semidefinite.

Autocovariance

The autocovariance of the n-element RV X is defined as

$$\begin{aligned} C_X &= E\left[\left(X - \bar{X}\right)\left(X - \bar{X}\right)^T\right] \\ &= \begin{bmatrix} E\left[\left(X_1 - \bar{X_1}\right)^2\right] & \cdots & E\left[\left(X_1 - \bar{X_1}\right)\left(X_n - \bar{X_n}\right)\right] \\ \vdots & \ddots & \vdots \\ E\left[\left(X_n - \bar{X_n}\right)\left(X_1 - \bar{X_1}\right)\right] & \cdots & E\left[\left(X_n - \bar{X_n}\right)^2\right] \end{bmatrix} \\ &= \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_{1n} \\ \vdots & \ddots & \vdots \\ \sigma_{n1} & \cdots & \sigma_n^2 \end{bmatrix} \end{aligned}$$

Since $\sigma_{ij} = \sigma_{ji}$, then $C_X = C_X^T$

Important

An autocovariance matrix is always symmetric

For any n-element column vector z, we have

$$\begin{aligned} z^T C_X z &= z^T E\big[(X-\bar{X})(X-\bar{X})^T\big] z \\ &= E\big[z^T (X-\bar{X})(X-\bar{X})^T z\big] \\ &= E\big[(z^T(X-\bar{X}))^2\big] \\ &\geq 0 \end{aligned}$$

Important

An autocovariance matrix is always positive semidefinite.

Gaussian RV

An n-element RV X is multivariate Gaussian with mean $\bar{X}$ and covariance C_X if its PDF is

$$f_X(x) = \frac{1}{(2\pi)^{\frac{n}{2}} |C_X|^{\frac{1}{2}}} e^{-\frac{1}{2}(X-\bar{X})^T C_X^{-1}(X-\bar{X})}$$

Linear transformation

Consider a Gaussian RV X undergoes a linear transformation

$$Y = g(X) = AX + b$$

Where A is a constant nxn matrix, and b is a constant n-element vector

If A is invertible, then

$$X = h(Y) = A^{-1}Y - A^{-1}b$$

Normality

We have

$$\begin{aligned} f_Y(y) &= |h'(y)| f_X\big[h(y)\big] \\ &= |A^{-1}| \frac{1}{(2\pi)^{\frac{n}{2}} |C_X|^{\frac{1}{2}}} e^{-\frac{1}{2}(A^{-1}y - A^{-1}b - \bar{x})^T C_X^{-1}(A^{-1}y - A^{-1}b - \bar{x})} \\ &= |A^{-1}| \frac{1}{(2\pi)^{\frac{n}{2}} |C_X|^{\frac{1}{2}}} e^{-\frac{1}{2}(A^{-1}y - A^{-1}b - A^{-1}A\bar{x})^T C_X^{-1}(A^{-1}y - A^{-1}b - A^{-1}A\bar{x})} \\ &= \frac{1}{(2\pi)^{\frac{n}{2}} |A||C_X|^{\frac{1}{2}}} e^{-\frac{1}{2}\left[A^{-1}y - A^{-1}\left(b - A\bar{x}\right)\right]^T C_X^{-1}\left[A^{-1}y - A^{-1}\left(b - A\bar{x}\right)\right]} \\ &= \frac{1}{(2\pi)^{\frac{n}{2}} A^{\frac{1}{2}} |C_X|^{\frac{1}{2}}} |A^T|^{\frac{1}{2}} e^{-\frac{1}{2}(A^{-1}y - A^{-1}\bar{y})^T C_X^{-1}(A^{-1}y - A^{-1}\bar{y})} \\ &= \frac{1}{(2\pi)^{\frac{n}{2}} |AC_X A^T|^{\frac{1}{2}}} e^{-\frac{1}{2}(y-\bar{y})^T\left(AC_X^{-1}A^T\right)(y-\bar{y})} \end{aligned}$$

$$y \sim N\left(A\bar{x} + b, AC_X A^T\right)$$

Important

Normality is preserved in linear transformation of random vectors.

1.2.4 Stochatsic processes

A stochastic process $X(t)$ is an RV X that changes with time.

Types of stochastic processes

- **Continuous random process:** is a random process that the RV at each time is continuous and time is continuous
- **Discrete random process:** is a random process that the RV at each time is discrete and time is continuous
- **Continuous random sequence:** is a random process that the RV at each time is continuous and time is discrete
- **Discrete random sequence:** is a random process that the RV at each time is discrete and time is discrete

PDF

The PDF of X is

$$F_X(x,t) = P\big[X(t) \leq x\big]$$

Important

If $X(t)$ is a random vector, then the inequality above is an element-by element inequality

CDF

The CDF of X is

$$f_X(x,t) = \frac{dF_X(x,t)}{dx}$$

Important

If $X(t)$ is a random vector, then the derivative above is taken once with respect to each element of x

Mean

The mean of X is

$$\bar{x}(t) = \int_{-\infty}^{\infty} x f(x,t) dx$$

Covariance

The covariance of X is

$$\begin{aligned} C_X(t) &= E\big[(X(t) - \bar{x}(t))(X(t) - \bar{x}(t))^T\big] \\ &= \int_{\mathbb{R}^n} \big[x - \bar{x}(t)\big]\big[x - \bar{x}(t)\big]^T f_X(x,t)\, dx \end{aligned}$$

Joint distribution

Second-order PDF

The second-order PDF of X is defined as

$$F(x_1, x_2, t_1, t_2) = P\big(X(t_1) \leq x_1, X(t_2) \leq x_2\big)$$

Important

If $X(t)$ is a random vector, then the inequality above consists of $2n$ inequalities

Second-order CDF

The second-oder CDF of X is

$$f(x_1, x_2, t_1, t_2) = \frac{\partial^2 F(x_1, x_2, t_1, t_2)}{\partial x_1 \partial x_2}$$

Important

If $X(t)$ is a random vector, the derivative above consists of $2n$ derivatives

Autocorrelation

The correlation between two RVs $X(t_1)$ and $X(t_2)$ is called the autocorrelation of the stochastic process $X(t)$

$$R_X(t_1, t_2) = E\big[X(t_1)X^T(t_2)\big]$$

Autocovariance

The autocovariance of a stochastic process is defined as

$$C_X(t_1, t_2) = E\big[\big(X(t_1) - \bar{X}(t_1)\big)\big(X(t_2) - \bar{X}(t_2)\big)^T\big]$$

Strict-sense stationary process: SSS

Strict-sense stationary (SSS) stochastic process is a stochastic process for which the PDF does not change with time.

Important

The mean of the stationary stochastic process is constant with respect to time, and the autocorrelation is a function of the time difference $t_2 - t_1$

$$\bar{x} = E\big[X(t)\big]$$
$$R_X(t_2 - t_1) = E\big[X(t_1)X^T(t_2)\big]$$

Wide-sense stationary process: WSS

Wide-sense stationary (WSS) stachastic process is a stochastic process for which the PDF changes with time, but the mean is constant with resepct to time and the autocorrelation is a function of the time difference.

From the definition of autocorrelation, it can be shown taht for WSS process the following propeties hold

$$R_X(0) = E\big[X(t)X^T(t)\big]$$
$$R_X(-\tau) = R_X(\tau)$$

Important

A stationary process is wide-sense srarionary, but a wide-sense stationary process may or may not be stationary

For scalar stichastic processes, it can be shown that

$$|R_X(\tau)| \leq R_X(0)$$

Ergodic process

Suppose we have a stochastic process $X(t)$

Suppose that the process has a realization $x(t)$

The time average of $X(t)$ is defined, in continuous-time random processes, as

$$A\big[X(t)\big] = \lim_{T\to\infty} \frac{1}{2T} \int_{-T}^{T} x(t)dt$$

The time autocorrelation of $X(t)$ is defined, in continuouse-time random processes, as

$$R\big[X(t), \tau\big] = A\big[X(t)X^T(t+\tau)\big]$$

Important

Discrete-time random processes are straightforward extensions of the continuous-timer random processes

An ergodic process is a stationary random process for which

$$E\big(X\big) = A\big[X(t)\big]$$
$$R_X(\tau) = R\big[X(t), \tau\big]$$

Important

If the random process is ergodic, then we can use those time averages to estimate the statistics of the stochastic process

Cross correlation and cross covariance

The cross correlation of $X(t)$ and $Y(t)$ is defined as

$$R_{XY}(t_1, t_2) = E\big[X(t_1)Y^T(t_2)\big]$$

Two random processes $X(t)$ and $Y(t)$ are said to be uncorrelated if $R_{XY}(t_1, t_2) = E\big[X(t_1)\big]E\big[Y^T(t_2)\big]$ for all t_1 and t_2

The cross covariance of $X(t)$ and $Y(t)$ is defined as

$$C_{XY}(t_1, t_2) = E\big[\big(X(t_1) - \bar{X}(t_1)\big)\big(Y(t_2) - \bar{Y}(t_2)\big)^T\big]$$

1.2.5 White noise and colored noise

If the RV $X(t_1)$ is independent from the RV $X(t_2)$ for all $t_1 \neq t_2$, then $X(t)$ is called white noise; otherwise, $X(t)$ is called colored noise

The power spectrum $S_X(\omega)$ of a wide-sense stationaru stochastic process $X(t)$ is defined as the Fourier trnasform os the autocorrelation.

$$S_X(\omega) = \int_{-\infty}^{\infty} R_X(\tau) e^{-j\omega\tau} d\tau$$
$$R_X(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_X(\omega) e^{j\omega\tau} d\omega$$

These equations are called the Wiener-Khintchine relations after Norbert Wiener anf Aleksandr Khinchin.

The power of a wide-sense stationaru stochastic process is defined as

$$P_X = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_X(\omega) d\omega$$

The cross correlation of a wide-sense stationary stochastic process is the inverse Fourier transform of the cross power spectrum of two wide-sense stationary stochastic processes $X(t)$ and $Y(t)$

$$S_{XY}(\omega) = \int_{-\infty}^{\infty} R_{XY}(\tau) e^{-j\omega\tau} d\tau$$
$$R_{XY}(\tau) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_{XY}(\omega) d^{j\omega\tau} d\omega$$

The power spectrum of a discrete-time random process is defined as

$$S_X(\omega) = \sum_{k=-\infty}^{\infty} R_X(k) e^{-j\omega k}, \omega \in [-\pi, \pi]$$
$$R_X(k) = \frac{1}{2\pi} \int_{-\infty}^{\infty} S_X(\omega) e^{j\omega k} d\omega$$

A discrete-time stochastic process $X(t)$ is called wite noise if

$$R_X(k) = \begin{cases} \sigma^2, k = 0 \\ 0, k \neq 0 \end{cases}$$

where δ_k is the kronecker delta function, defined as

$$\delta_k = \begin{cases} 1, k = 0 \\ 0, k \neq 0 \end{cases}$$

If $X(t)$ is a dicrete-time white noise process, then the RV $X(n)$ is uncorrelated with $X(m)$ unless $m = n$

The power of a discrete-time white noise process is equal at all frequencies

$$S_X(\omega) = R_X(0), \omega \in [-\pi, \pi]$$

For a continuous-time random process, white noise has equal power at all frequencies

$$S_X(\omega) = R_X(0)$$

Substitute this expression for $S_X(\omega)$ into $R_X(\tau)$, then for continuous-time white noise

$$R_X(\tau) = R_X(0)\delta(\tau)$$

where $\delta(\tau)$ is the continuous-time impulse function

Example

Suppose that a zero-mean stationary stochsatic process has the autocoreelation function

$$R_X(\tau) = \sigma^2 e^{-\beta|\tau|}$$

where β is a positive real number

The power spectrum is computed as

$$\begin{aligned} S_X(\omega) &= \int_{-\infty}^{\infty} \sigma^2 e^{-\beta|\tau|} e^{-j\omega\tau} d\tau \\ &= \int_{-\infty}^{0} \sigma^2 e^{(\beta - j\omega)\tau} d\tau + \int_{0}^{\infty} \sigma^2 e^{-(\beta + j\omega)\tau} d\tau \\ &= \frac{\sigma^2}{\beta - j\omega} + \frac{\sigma^2}{\beta + j\omega} \\ &= \frac{2\sigma^2\beta}{\omega^2 + \beta^2} \end{aligned}$$

The variance of stochastic process is computed as

$$\begin{aligned} E\left[X^2(t)\right] &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \frac{2\sigma^2\beta}{\omega^2 + \beta^2}\, d\omega \\ &= \frac{\sigma^2\beta}{\pi} \left[\frac{1}{\beta} \arctan\left(\frac{\omega}{\beta}\right)\right]_{-\infty}^{\infty} \\ &= \sigma^2 \\ &= R_X(0) \end{aligned}$$

1.3 Least Square Estimation

CHAPTER

TWO

KALMAN FILTER

2.1 Kalman Filter Implementation

With continuouse state-space model state-space equation:

$$\dot{x} = Ax + Bu + \sqrt{Q_C}v(t)$$
$$y = Cx + \sqrt{R}w(t)$$

Discretization the model into:

$$x_k = x_{k-1} + T_s(Ax_{k-1} + Bu_{k-1}) + \sqrt{Q_c T_s}\, v_{k-1}$$
$$y_k = Cx_k$$

```
c = [1, 0]

function x_dot = condyn(x, u, A, B)
    x_dot = A * x + B * u
end

function x_k = disdyn(x, u, A, B, T_s, Q_c)
    n = length(x);
    x_k = x + T_s * condyn(x, u, A, B) + \sqrt(Q_c * T_s) * random(n, 1)
end

function y = measure(x, c)
    y = c * x
end
```

Note

condyn = continuous dynamic

disdyn = discrete dynamic

A = transition matrix of continuous state

B = input matrix of continuous state

```
for k = 2:N_sample
    % true state anf measurement
    u = sin(t(k - 1));
    [xk, ~] = disdyn(x_est_store(:, k - 1), u, A, B, T_s);
    x_store(:, k) = xk + sqrt(Q_c * T_s) * randn(2, 1);
    y_store(1, k) = measure(x_store(:, k), c) + sqrt(R) * randn(1, 1);
```

(continues on next page)

(continued from previous page)

```
    % kalman filter
    % next state prediction and error covariance update
    [xk, Ak] = disdyn(x_est_store(:, k - 1), u, A, B, T_s);
    x_est_store(:, k) = xk;
    p = Ak * p * Ak' + Q_d;

    % estimate measurement, cross covariance, auto covariance, kalman gain
    y_est = measure(x_est_store(:, k), c);
    p_xy = p * c;
    p_yy = c * p * c' + r; % innovation covariance
    k = p_xy/p_yyy; % kalman gain
    x_est_store(:, k) = x_est_store(:, k) + k *y_store(1, k) - y_est); % update␣
↪estimate
    p = (eye(2) - k * c) + p; update error covariance
end
```

2.1.1 Python implementation

```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# Parameters
# ----------------------------
Ts = 0.1           # sampling time
N_sample = 100     # number of samples

A = np.array([[0, 1],
              [0, 0]])  # continuous-time A
B = np.array([[0],
              [1]])     # continuous-time B
c = np.array([[1, 0]])  # measurement matrix

Q_c = np.array([[0.01, 0],
                [0, 0.01]])  # continuous process noise
R = 0.1  # measurement noise

Q_d = Q_c * Ts  # discrete process noise

# ----------------------------
# Storage initialization
# ----------------------------
x_store = np.zeros((2, N_sample))
y_store = np.zeros(N_sample)
x_est_store = np.zeros((2, N_sample))
p = np.eye(2)  # initial error covariance

x_store[:, 0] = [0, 0]
x_est_store[:, 0] = [0, 0]

t = np.arange(N_sample) * Ts

# ----------------------------
# Functions
# ----------------------------
```

(continues on next page)

(continued from previous page)

```
def condyn(x, u, A, B):
    """Continuous dynamics"""
    return A @ x + B.flatten() * u

def disdyn(x, u, A, B, Ts):
    """Discrete dynamics and linearized Ak"""
    xk = x + Ts * condyn(x, u, A, B)
    Ak = np.eye(len(x)) + Ts * A  # linearized discrete-time transition
    return xk, Ak

def measure(x, c):
    """Measurement function"""
    return c @ x

# ----------------------------
# Simulation loop
# ----------------------------
for k in range(1, N_sample):
    u = np.sin(t[k-1])

    # True system + measurement noise
    xk, _ = disdyn(x_est_store[:, k-1], u, A, B, Ts)
    x_store[:, k] = xk + np.random.multivariate_normal([0,0], Q_c*Ts)
    y_store[k] = measure(x_store[:, k], c) + np.random.normal(0, np.sqrt(R))

    # Kalman filter prediction
    xk_pred, Ak = disdyn(x_est_store[:, k-1], u, A, B, Ts)
    x_est_store[:, k] = xk_pred
    p = Ak @ p @ Ak.T + Q_d

    # Measurement update
    y_pred = measure(x_est_store[:, k], c)
    p_xy = p @ c.T
    p_yy = c @ p @ c.T + R
    K = p_xy / p_yy  # Kalman gain
    x_est_store[:, k] = x_est_store[:, k] + K.flatten() * (y_store[k] - y_pred)
    p = (np.eye(2) - K @ c) @ p

# ----------------------------
# Plot results
# ----------------------------
plt.figure(figsize=(10,6))
plt.plot(t, x_store[0, :], label='True position')
plt.plot(t, x_est_store[0, :], label='Estimated position', linestyle='--')
plt.plot(t, y_store, label='Measurements', linestyle=':', alpha=0.7)
plt.xlabel('Time [s]')
plt.ylabel('Position')
plt.title('Kalman Filter Tracking (Updated Implementation)')
plt.legend()
plt.grid(True)
plt.show()
```

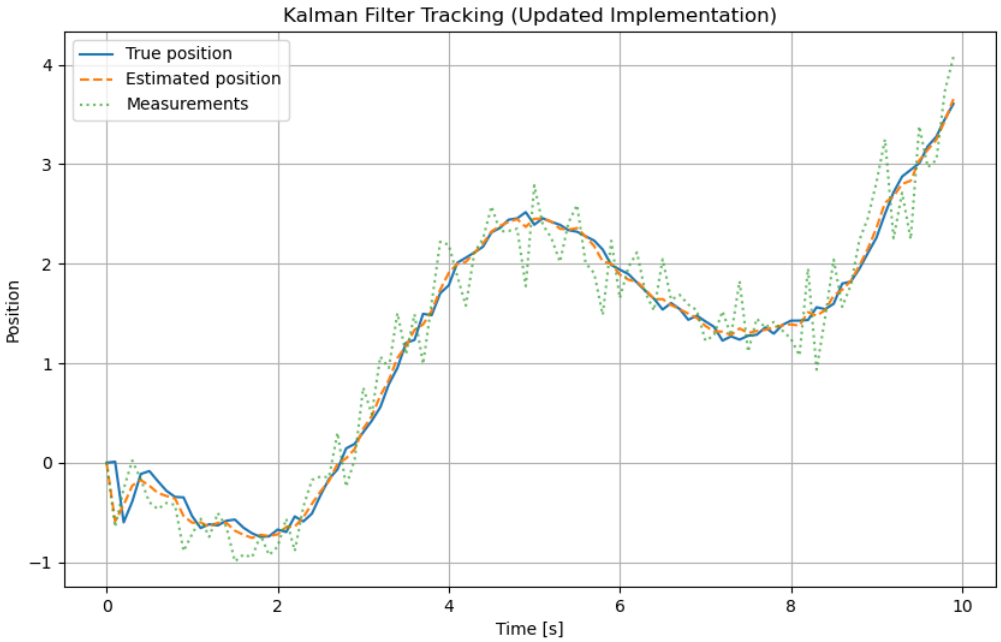


Fig. 1: Kalman filter simulation results: true state, measurements, and estimated state.

2.2 Extended Kalman Filter

2.2.1 Review (Kalman filter) on covariance prediction and correction

For a nonlinear dynamic system:

$$x_k = f\big(x_{k-1},\, u_{k-1}\big) \quad \text{(nonlinear state transition)}$$
$$y_k = g\big(x_k,\, u_k\big) \quad \text{(nonlinear measurement)}$$

- Time update (prediction):

$$P_{k|k-1} = A_k\, P_{k-1|k-1}\, A_k^T + Q_d$$

Note: A constant linear matrix A does not exist for nonlinear systems; instead we linearize about the current estimate and use a time-varying Jacobian A_k.

- Measurement (correction):

$$P_{k|k} = \big(I - K_k\, C_k\big)\, P_{k|k-1}$$

The key step in the Extended Kalman Filter (EKF) is computing the Jacobians A_k and C_k (the linearizations of f and g about the current estimate):

$$A_k = \left.\frac{\partial f}{\partial x}\right|_{x=\hat{x}_{k-1|k-1}}, \qquad C_k = \left.\frac{\partial g}{\partial x}\right|_{x=\hat{x}_{k|k-1}}$$

The Jacobian A_k can be written elementwise as

$$A_k = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{x=\hat{x}_{k-1|k-1}}$$

and similarly for C_k using partial derivatives of g.

and similarly for C_k using partial derivatives of g (measurements $y \in \mathbb{R}^m$):

$$C_k = \begin{bmatrix} \frac{\partial g_1}{\partial x_1} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \cdots & \frac{\partial g_m}{\partial x_n} \end{bmatrix}_{x=\hat{x}_{k|k-1}}$$

For differential equations (continuous $\rightarrow$ discrete):

$$\dot{x} = f(x,u) \quad \Rightarrow \quad x_k \approx x_{k-1} + T_s\, f\big(x_{k-1},\, u_{k-1}\big)$$

The discrete-time Jacobian (linearized about the estimate) is:

$$A_{k-1} = I + T_s \left.\frac{\partial f}{\partial x}\right|_{x=\hat{x}_{k-1|k-1}}$$

The estimated state vector (column form) is written as:

$$\hat{x}_{k|k} = \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \vdots \\ \hat{x}_n \end{bmatrix}$$

Example

$$\dot{\omega} = -a\,\omega + 1.5\,\sin(t) + b + \sqrt{0.1}\,v(t)$$

Let

$$x_1 = \omega, \quad x_2 = a, \quad x_3 = b$$

Then the continuous-time dynamics for the state components are

$$\begin{aligned} \dot{x}_1 &= -x_2\,x_1 + 1.5\,\sin(t) + x_3 + \sqrt{0.1}\,v_1(t) \\ \dot{x}_2 &= 0 + \sqrt{0.01}\,v_2(t) \\ \dot{x}_3 &= 0 + \sqrt{0.01}\,v_3(t) \end{aligned}$$

or in compact form

$$\dot{x} = f(x,t) + \sqrt{Q_c}\,v(t)$$

with

$$f(x,t) = \begin{bmatrix} -x_2\,x_1 + 1.5\,\sin(t) + x_3 \\ 0 \\ 0 \end{bmatrix}$$

and the process-noise term

$$\sqrt{Q_c}\,v(t) = \begin{bmatrix} \sqrt{0.1} & 0 & 0 \\ 0 & \sqrt{0.01} & 0 \\ 0 & 0 & \sqrt{0.01} \end{bmatrix} \begin{bmatrix} v_1(t) \\ v_2(t) \\ v_3(t) \end{bmatrix}$$

The continuous-time Jacobian:

$$A_c = \partial f/\partial x = \begin{bmatrix} -x_2 & -x_1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The discretized linearization (Euler, sample time T_s) is

$$A_{k-1} = I_3 + T_s\,A_c\big|_{x=\hat{x}_{k-1|k-1}}$$

If we measure only the angular rate ω, the measurement model is

$$y = \omega = x_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}\,x$$

```
% ekf_example.m
% Extended Kalman Filter example (3-state) using Euler discretization
function ekf_example()
    Ts = 0.1;
    N = 200;
    t = 0:Ts:Ts*(N-1);

    % true initial state
    x_true = [0.5; 0.1; 0.2];
    % initial estimate
    x = [0;0;0];
    P = diag([0.5,0.5,0.5]);

    Qc = diag([0.1, 0.01, 0.01]);
    R = 0.05;

    rng('default');

    for k = 1:N
        tk = t(k);
        % simulate true system (Euler)
        w = mvnrnd([0 0 0], Qc)';
        x_true = x_true + Ts * f_cont(x_true, tk) + sqrt(Ts)*w;

        % measurement
        v = sqrt(R)*randn();
        y = x_true(1) + v;

        % predict
        [x_pred, P_pred] = ekf_predict_matlab(x, P, Ts, Qc, tk);

        % update
        [x, P] = ekf_update_matlab(x_pred, P_pred, y, R);
    end

    disp('Final true state:'); disp(x_true');
    disp('Final estimate:'); disp(x');
end

% Plot results
figure;
tt = 0:Ts:Ts*(N-1);
subplot(3,1,1);
plot(tt, true_states(1,:),'b-', tt, est_states(1,:),'r--');
legend('true omega','est omega'); ylabel('omega');
subplot(3,1,2);
plot(tt, true_states(2,:),'b-', tt, est_states(2,:),'r--');
legend('true a','est a'); ylabel('a');
subplot(3,1,3);
plot(tt, true_states(3,:),'b-', tt, est_states(3,:),'r--');
legend('true b','est b'); ylabel('b'); xlabel('time (s)');


function dx = f_cont(x, t)
    omega = x(1); a = x(2); b = x(3);
    dx = [-a*omega + 1.5*sin(t) + b; 0; 0];
end
```

(continues on next page)

(continued from previous page)

```
function A = A_cont(x, t)
    omega = x(1); a = x(2);
    A = [-a, -omega, 1; 0,0,0; 0,0,0];
end

function [x_pred, P_pred] = ekf_predict_matlab(x, P, Ts, Qc, t)
    x_pred = x + Ts * f_cont(x, t);
    A = eye(3) + Ts * A_cont(x, t);
    P_pred = A * P * A' + Ts * Qc;
end

function [x_upd, P_upd] = ekf_update_matlab(x_pred, P_pred, y, R)
    C = [1 0 0];
    S = C * P_pred * C' + R;
    K = (P_pred * C') / S;
    x_upd = x_pred + K * (y - C * x_pred);
    P_upd = (eye(3) - K * C) * P_pred;
end
```

```
import numpy as np

def f(x, t):
    # continuous dynamics converted to discrete via Euler (sample time applied␣
↪outside)
    # x = [omega, a, b]
    omega, a, b = x
    dx1 = -a * omega + 1.5 * np.sin(t) + b
    dx2 = 0.0
    dx3 = 0.0
    return np.array([dx1, dx2, dx3])

def g(x):
    # measurement: y = omega
    return np.array([x[0]])

def A_continuous(x, t):
    # Jacobian of f wrt x (continuous-time)
    omega, a, b = x
    return np.array([[-a, -omega, 1.0],
                     [0.0, 0.0, 0.0],
                     [0.0, 0.0, 0.0]])

def C_jacobian(x):
    # Jacobian of g wrt x
    return np.array([[1.0, 0.0, 0.0]])

def ekf_predict(x, P, Ts, Qc, t):
    # discrete-time predict using Euler discretization
    # x_{k} = x_{k-1} + Ts * f(x_{k-1}, t)
    x_pred = x + Ts * f(x, t)
    A = np.eye(3) + Ts * A_continuous(x, t)
    P_pred = A @ P @ A.T + Ts * Qc
    return x_pred, P_pred

def ekf_update(x_pred, P_pred, y, R):
```

(continues on next page)

(continued from previous page)

```
    C = C_jacobian(x_pred)
    S = C @ P_pred @ C.T + R
    K = P_pred @ C.T @ np.linalg.inv(S)
    x_upd = x_pred + (K @ (y - g(x_pred))).reshape(-1)
    P_upd = (np.eye(len(x_pred)) - K @ C) @ P_pred
    return x_upd, P_upd

def simulate_and_run(Ts=0.1, N=200):
    t = 0.0
    # true initial state
    x_true = np.array([0.5, 0.1, 0.2])
    # initial estimate
    x = np.array([0.0, 0.0, 0.0])
    P = np.diag([0.5, 0.5, 0.5])

    # continuous process noise covariance (Qc) and measurement noise (R)
    Qc = np.diag([0.1, 0.01, 0.01])
    R = np.array([[0.05]])

    history = []

    for k in range(N):
        # simulate true system (Euler discrete approx)
        w = np.random.multivariate_normal(np.zeros(3), Qc) * np.sqrt(Ts)
        x_true = x_true + Ts * f(x_true, t) + w

        # measurement
        v = np.random.normal(0, np.sqrt(R[0,0]))
        y = np.array([x_true[0] + v])

        # EKF predict
        x_pred, P_pred = ekf_predict(x, P, Ts, Qc, t)

        # EKF update
        x, P = ekf_update(x_pred, P_pred, y, R)

        history.append((t, x_true.copy(), x.copy(), P.copy(), y.copy()))
        t += Ts

    return history

if __name__ == '__main__':
    hist = simulate_and_run()
    # print final results
    t, x_true, x_est, P, y = hist[-1]
    print('Final time: {:.2f}s'.format(t))
    print('True state: ', x_true)
    print('Estimated:  ', x_est)
    print('Measurement: ', y)

    # Prepare and save plots
    try:
        import matplotlib
        matplotlib.use('Agg')
        import matplotlib.pyplot as plt
```

(continues on next page)

(continued from previous page)

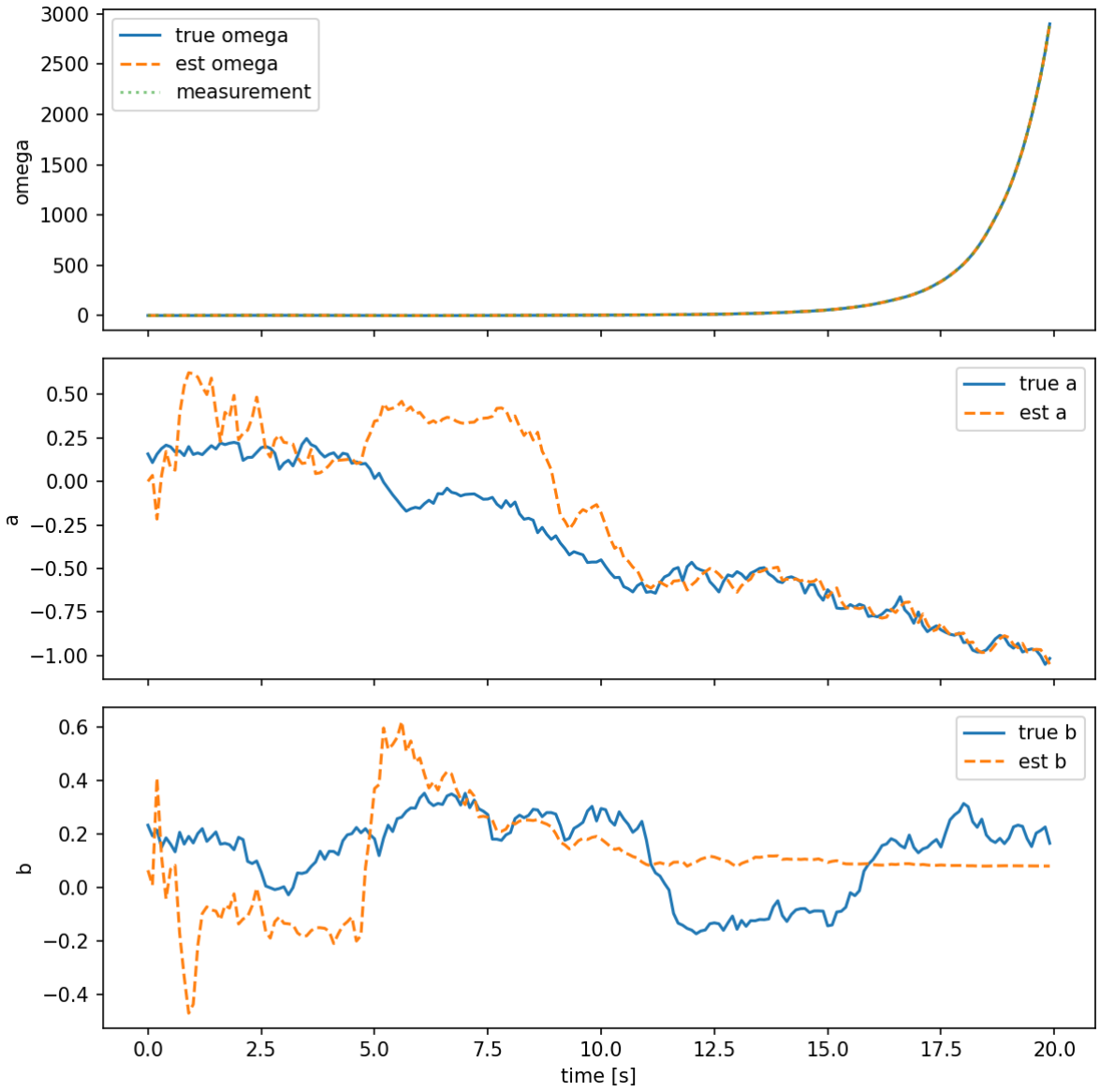
```
        times = np.array([h[0] for h in hist])
        true_vals = np.vstack([h[1] for h in hist])
        est_vals = np.vstack([h[2] for h in hist])
        meas = np.hstack([h[4][0] for h in hist])

        fig, ax = plt.subplots(3, 1, figsize=(8, 8), sharex=True)
        ax[0].plot(times, true_vals[:,0], label='true omega')
        ax[0].plot(times, est_vals[:,0], label='est omega', linestyle='--')
        ax[0].plot(times, meas, label='measurement', linestyle=':', alpha=0.6)
        ax[0].legend()
        ax[0].set_ylabel('omega')

        ax[1].plot(times, true_vals[:,1], label='true a')
        ax[1].plot(times, est_vals[:,1], label='est a', linestyle='--')
        ax[1].legend()
        ax[1].set_ylabel('a')

        ax[2].plot(times, true_vals[:,2], label='true b')
        ax[2].plot(times, est_vals[:,2], label='est b', linestyle='--')
        ax[2].legend()
        ax[2].set_ylabel('b')
        ax[2].set_xlabel('time [s]')

        plt.tight_layout()
        out_path = 'ekf_plot.png'
        fig.savefig(out_path, dpi=150)
        print(f'Plot saved to {out_path}')
    except Exception as e:
        print('Plotting failed:', e)
```


true omega
est omega
measurement
omega
3000
2500
2000
1500
1000
500
0
true a
est a
a
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
true b
est b
b
0.6
0.4
0.2
0.0
−0.2
−0.4
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
time [s]